

Part A: Conceptual Foundation (Theory)

Q1. What is Regularization in Machine Learning? Why is it needed?

Regularization is a technique used to reduce overfitting in machine learning models by adding a penalty term to the loss function. This penalty discourages the model from learning overly complex patterns that fit the training data too closely but fail to generalize to new data.

Why it is needed:

- Without regularization, a model may memorize training data (overfitting), leading to very high training accuracy but poor test accuracy.
- It controls model complexity by shrinking or eliminating less important feature coefficients.
- It improves generalization — meaning the model performs well on unseen data too.

Q2. Difference between Ridge Regression (L2) and Lasso Regression (L1)

Aspect	Ridge (L2)	Lasso (L1)
Penalty Term	Sum of squares of coefficients (β^2)	Sum of absolute values of coefficients (β)
Effect on Coefficients	Shrinks toward zero, never exactly zero	Can shrink coefficients to exactly zero
Feature Selection	No — keeps all features	Yes — eliminates irrelevant features
Best Used When	All features are relevant	Many irrelevant/redundant features exist
Formula	$\text{Loss} + \alpha \cdot \sum \beta^2$	$\text{Loss} + \alpha \cdot \sum \beta$

Q3. What is Cross-Validation and why is it important?

Cross-Validation (CV) is a model evaluation technique where the dataset is split into multiple subsets called folds. The model is trained on some folds and validated on the remaining fold. This process repeats until every fold has served as the test set once. The final performance score is the average across all folds.

Why it is important:

- Provides a more reliable estimate of model performance than a single train/test split.
- Reduces the risk of overfitting to any particular data split.
- Helps in selecting the best hyperparameters (e.g., best α value for Ridge or Lasso).
- Makes better use of limited data by training and validating on the entire dataset.

Q4. Cross-Validation Techniques

a) K-Fold Cross-Validation

The dataset is divided into K equal-sized folds. The model is trained on K-1 folds and tested on the remaining 1 fold. This process repeats K times so that every fold serves as the test set exactly once. The final score is the average of all K scores. Example: In 5-Fold CV, data is split into 5 parts and 5 models are trained.

b) Stratified K-Fold Cross-Validation

Similar to K-Fold, but ensures that each fold maintains a proportional representation of the target variable's distribution. It is particularly useful for classification problems with imbalanced classes or regression problems with a skewed target distribution. It prevents any single fold from having a very different target distribution compared to others.

c) Leave-One-Out Cross-Validation (LOOCV)

A special case of K-Fold where $K = N$ (total number of samples). In each iteration, the model trains on all samples except one, which is used for testing. This repeats N times. Advantage: Maximum use of available data. Disadvantage: Computationally very expensive for large datasets.

d) Time Series Split

Specifically designed for time-ordered data where future data must not be used to predict the past. In each split, the training set contains only past observations and the test set contains future observations. The training window grows with each fold. Example: Train on months 1-6, test on month 7; then train on months 1-7, test on month 8; and so on.

Q5. Why are Tree-Based Models Less Sensitive to Feature Scaling?

Tree-based models (Decision Tree, Random Forest, Gradient Boosting, etc.) are not sensitive to feature scaling because they make decisions based on **threshold splits** on individual features, not on distances or magnitudes between features.

Key reasons:

- A decision tree asks: "*Is feature $X > \text{threshold } t$?*" — the actual scale of X does not matter, only the relative order of values matters.
- If you multiply a feature by 1000, the tree will simply choose a threshold 1000× larger — the split logic remains unchanged.
- In contrast, linear models and SVMs use dot products and distances where large-valued features dominate, making scaling essential for them.
- Random Forest and other ensemble tree models inherit this same property since they are built from decision trees.